

Arquitetura de Computadores Modernos

Passos da execução de um programa:

1. Fetch (busca) da nova instrução da memória p/ o registro.
2. PC (program counter) aponta p/ nova instrução.
3. Determinação do tipo de instrução buscada em (1.)
4. Se a instrução usa uma palavra da memória, determinar aonde ela está
5. Se necessário, buscar a palavra na memória
6. Executar a instrução
7. Voltar p/ 1.

- Esses passos dependem da linguagem de máquina do processador. Esta linguagem pode ser implementada diretamente via "hardware" (componentes eletrônicos) ou via programa chamado interpretador que transforma essas instruções em um conjunto de instruções em um conjunto de instruções mais simples que podem ser executadas por um 'hardware' mais simples de ser construído.

- Instruções mais longas são mais "caras" para serem implementadas no hardware, porém quando implementadas provêm ganhos de performance:

- Busca de conjuntos mais poderosos, e mais complexos, de instruções.
 - instruções específicas p/ ponto flutuante
 - suporte direto ao acesso de elementos de matrizes
 - uma única instrução otimizada para substituir conjuntos de instruções que aparecem sempre em sequência.

- O custo para implementar via hardware estas instruções é alto, justificável apenas p/ high performance computer.

- O custo de manter computadores c/ conjuntos de instruções diferentes também é "alto". Programas para um computador não rodam em outro.

- Saída: Família de Processadores
 - High-End: instruções por hardware (conjunto completo)
 - Low-End: interpretador p/ transformar o conjunto completo de instruções em instruções mais simples p/ um hardware mais simples.

- IBM introduz o termo "Arquitetura" p/ descrever este nível de compatibilidade.

- Vantagens do computador com instruções interpretadas:

- Habilidade p/ corrigir instruções incorretamente implementadas, ou ainda, para mascarar deficiência de design no hardware básico.
- Possibilidade de incluir novas instruções a um custo mínimo, mesmo depois da entrega do computador.
- Design estruturado permite desenvolvimento, teste e documentação eficientes das instruções complexas.

- Nos anos 70 praticamente todos os computadores eram interpretados.
 - High end:
 - DEC VAX - várias centenas de instruções
 - Low end:
 - 780, Motorola 6800 (processadores de 8 bits com instruções interpretadas).
- Detalhes técnicos favoreciam também a interpretação:
 - Control stores - ROM's rápidas usadas p/ armazenar as microinstruções.
- Um motorola 6800 interpretava 1 microinstruções em 100 ns, e uma instrução típica tinha 10 microinstruções. Já o acesso à memória durava 500 ns.
 - Execução direta: 1 μ s (1 MHz)
 - Execução interpretada: 2 μ s (boa relação custo benefício)
 - Execução de 10 microinstruções: 6 μ s

RISC x CISC

1980 - Berkeley: Patterson e Sequin - RISC (SPARC)

1981 - Stanford: Hennessy - MIPS (MIPS)

- Em fase inicial: instruções simples p/ serem executadas rapidamente.
- Transformou-se em: projetar instruções que poderiam ser iniciadas rapidamente.

"Quão longa é uma instrução é um fator menos importante do que quantas instruções podem ser iniciadas por segundo."

Primeiras máquinas RISC	X	CISC na época (VAX)
50 instruções		200-300 instruções

RISC - Reduced Instruction Set Computer

CISC - Complex Instruction Set Computer

- Se uma instrução RISC for 10 vezes mais rápidas que uma CISC e cinco instruções RISC fizerem uma CISC, então RISC tem melhor desempenho.

Por que não temos hoje apenas máquinas RISC?

- Compatibilidade com programas p/ máquinas CISC anteriores (Intel)
- Núcleo RISC de novos processadores Intel (486 em diante). Interpreta apenas instruções menos comuns (mais complexas).

Princípios de design (projeto) para Computadores Modernos:

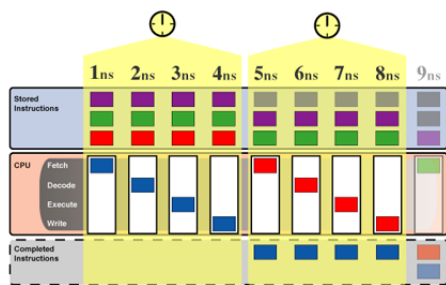
- Todas as instruções são executadas pelo hardware
 - Se o computador implementar instruções CISC, estas serão quebradas em partes separadas e então executadas como uma sequência de microinstruções.
 - O passo de quebra de instrução CISC em instruções RISC diminui a performance, mas para instruções menos frequentes pode ser aceitável.
- As instruções devem ser fáceis de decodificar
 - Um limite crítico na taxa de instruções iniciadas é a decodificação das instruções individuais e a determinação dos recursos que elas necessitam.
 - Tornar as instruções regulares, fixar seus tamanhos e reduzir o número de campos.
 - Quanto menos formatos existirem melhor.
- Somente LOADS e STORES devem referenciar a memória
 - Uma das mais simples maneiras de quebrar operações em passos mais simples é carregar e descarregar os operandos de registros, deixando as operações entre o registro e a memória para serem executadas em passos separados.
 - Operações entre a memória e os registros podem então ser executadas enquanto outras operações são executadas.
- Existência de um grande número de registros
 - Manter os dados nos registros é uma maneira de evitar a demora na busca desses dados na memória.

Paralelismo

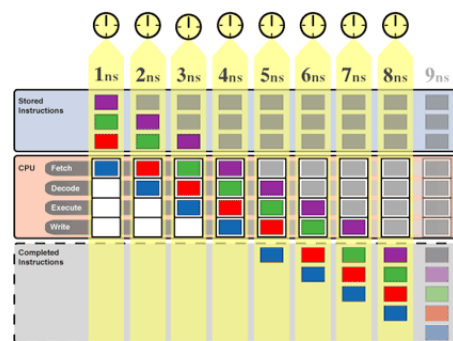
- Aumentar a velocidade do relógio é apenas uma maneira de melhorar a performance. Além disto, implica em busca de tecnologia de execução e em limites físicos para cada geração de novas tecnologias.
- Paralelismo, ou executar mais de uma instrução por vez, é uma maneira de garantir velocidade sem necessariamente aumentar a velocidade do relógio.
- O paralelismo no design de computadores apresenta-se em duas formas gerais:
 - Paralelismo a nível de instrução
 - Paralelismo a nível de processadores

Paralelismo a nível de Instruções

- Prefetch buffer (IBM - 1959) - buscava as instruções na memória antes de serem necessárias e armazenava num buffer (registro). Dividia a execução em dois passos: busca e execução.
- Pipeline vai mais além: divide a execução da instrução em várias partes, cada uma comandada por uma parte dedicada do hardware, possibilitando suas execuções em paralelo.

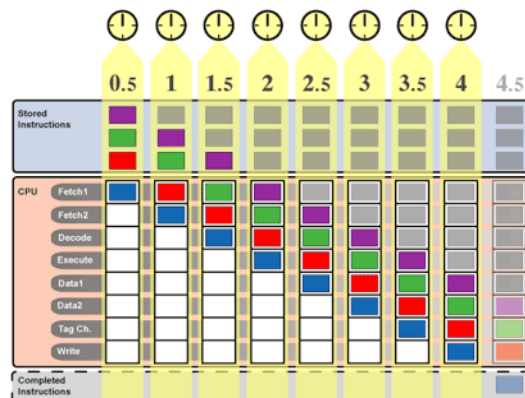


Ciclo de execução com 4 passos
(ArsTechnica)



Pipeline (linha de montagem)
(ArsTechnica)

- 10 passos num pipeline, 1 instrução 10 x unidades de relógio
- 100 instruções a serem executadas, 110 unidades de relógio
- diminuindo a latência (a instrução talvez pudesse ser executada num tempo melhor).
- aumentando a largura de banda (muito mais instruções por unidade de tempo).

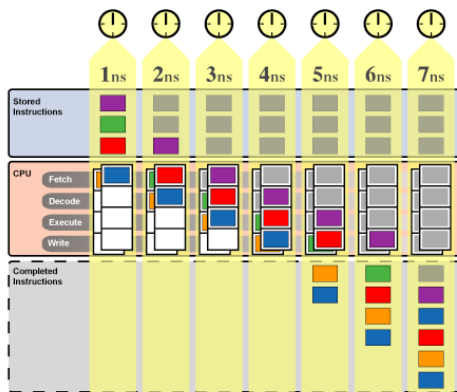


Pipeline de 8 passos
(ArsTechnica)

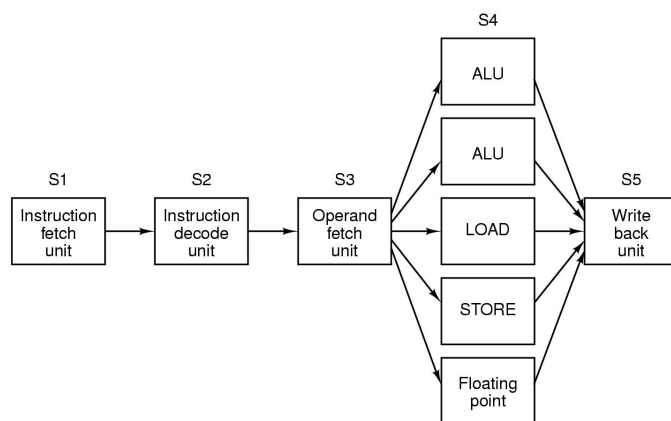
Arquitetura Superscalar

- Duplo Pipeline: uma instrução de fetch busca um par de instruções executadas em paralelo. As duas instruções não devem conflitar na busca de recursos. (Fig. 2.5)

- 486 usava 1 pipeline
- Pentium 2 pipelines: 1 para qualquer instrução (*u* pipeline) e para instruções inteiros simples, e um de floating point (*v* pipeline).
- Regras complexas definiam o uso dos dois pipelines: se as 2ª instruções não eram infinitamente simples para ser executadas no 2º pipeline, ela era alinhada com a próxima instrução e aí executada. Programas com números inteiros compilados especialmente para o Pentium rodavam em até metade do tempo do mesmo programa compilado para um 486 de mesmo clock.



Pipeline Superscalar (2 threads)
(ArsTechnica)



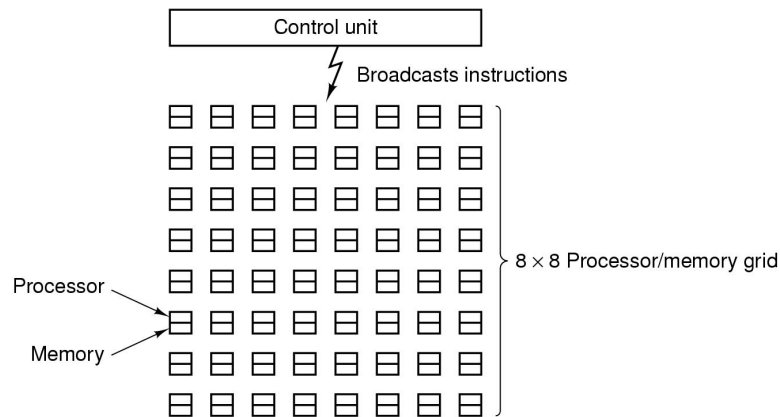
Pipeline Superscalar (múltiplas execuções)
(Tanenbaum - fig 2.6)

- Indo além de dobrar o pipeline.

- 4 pipelines implica em muitas regras de conduta e portanto muito mais complicado. No entanto, a execução das instruções, em princípio, leva mais tempo do que a busca e a decodificação. Então, se tivermos diversos mecanismos diferentes para o passo de execução, o computador não tem necessariamente de esperar pela conclusão desta instrução, ele pode passar adiante, e se a próxima instrução não depende desta, começar sua execução antes do término da primeira.

Paralelismo a nível de Processador

- Existe um limite para ganho desempenho com a pipeline ou com a arquitetura superescalar.
- Um grande número de processadores que executa a mesma sequência de instruções em diferentes conjuntos de dados. (Controle Central)



Array Computers (Tanenbaum fig. 2.7)

- Vector Processor: a mesma idéia de operar sobre muito dados de uma vez, porém a matriz de dados é colocada num registro vetorial, um conjunto de registros contíguos mais que podem ser alocados com um única instrução de maneira sequencial. Este vetor é então inserido num pipeline, que executa a instrução e armazena o resultado em outro registrador vetorial.

- Exemplos de computadores vetoriais CRAY, NEC.

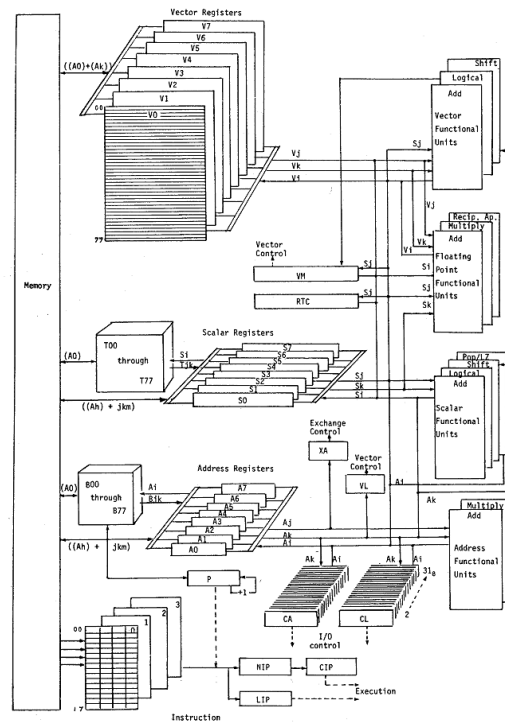
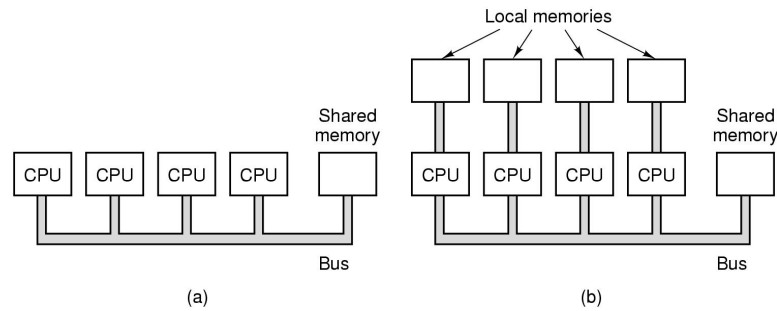


Figure 3-1. Computation section

Arquitetura de um computador Cray

<http://chrisfenton.com/homebrew-cray-1a/>

Multiprocessadores



Multiprocessadores (Tanenbaum - fig. 2.8)

- 1 única área de memória para diversas CPU's
- diferente do Array Computer por serem cpu's independentes
- software deve controlar o acesso à memória de cada um dos processadores
- Processadores rápidos acessando a mesma memória podem causar constantemente conflito no BUS. As soluções podem ser a utilização de memórias locais não acessíveis aos outros processadores. Esta memória pode ser usada para armazenar o código e dados que não são compartilhados.
- O compartilhamento de memória facilita a programação pois o programador não precisa se preocupar com a presença dos dados na memória de cada processador. Todos os dados estão na mesma memória.
- Exemplo: Análise de uma fotografia de uma célula. A foto está armazenada na memória e cada processador analisa um pedaço. Se houver superposição de pedaços, não é necessário transmitir essa informação para a memória do processador. Se uma segunda foto for analisada e os pontos a serem observados não coincidirem com os da 1ª foto é só o processador buscar a nova posição a ser observada na memória.

Multicomputadores

Conectar um grande número de processadores à memória pode ser extremamente difícil.

- Solução conectar computadores entre si, sem compartilhar a memória. Cada processador (ou grupo deles) contará com uma memória própria. Os dados necessários são transmitidos de um computador para outro através de protocolos de comunicação via rede, e diversas topologias são utilizadas:
 - malha 2D
 - malha 3D
 - estrela
 - anel
- Multicomputadores são mais fáceis de construir que multiprocessadores, porém multiprocessadores são mais fáceis de programar
- Tentativas de solução híbrida.